★ Binary Searching:

2	4	6	8	10	12	14
0	1	2	3	4	5	6

→ We are given by an array of integers which can be sorted in either ascending & descending order.

→ We are given by a [target] element or [value], we need to find whether exists in array or not!

→ This can be done by pointing 3 indexes of that array.

① We need to find the start of the array & point it using 'start' variable.

② We need to find its end & point it through 'end' variable.

③ We need to calculate the mid point of the array, its done by adding the start & end index & divide by 2 & store it in mid variable.

Suppose, here in the given array,

- start = 0, end = arr.length-1 (i.e. 6)
  target = 12,

You can find the middle element of index similarly by using the following formula:

[(start + end)/2]

→ But it can be problematic when you are dealing with too big arrays as:

(start + end) can lead to the exceeding of an integer limit.

→ e.g. integer in Java has a range (a limit of integer) is:
$(-2^{31}$ to $2^{31}-1)$

Or you can simply say us:
−2,147,483,648 to 2,147,483,647

→ Suppose in our binary search, our middle element at current index is 1,999,999,999 & the target value is bigger than value at that index we need to move our start by "+1"

→ So, start becomes = 2,000,000,000
(suppose:) our end is = 2,100,000,000,

→ if we are using "(start+end)/2", the (start + end) exceeds the 'int' limits.

→ but if we do this:
start + (end − start)/2 will not exceeds!

2,000,000,000 + (2,100,000,000 − 2,000,000,000)/2

2,000,000,000 + (100,000,000)/2

2,000,000,000 + 50,000,000

2,050,000,000 < Integer.MAX_VALUE

that's why we have this optimized formula & we need to use it in Program: start + (end - start)/2

suppose,

start → s
end → e

$s + \frac{e-s}{2}$

$\frac{s+e-s}{2}$

$\left(\frac{s+e}{2}\right)$ "same same"

Now comes again to binary search.

We have to run a loop until the ~~start~~ starting index is ~~not~~ exceeding the ending index, i.e. (start < end)

because sometimes the case becomes where the target value will be found when the start == end == mid

suppose, array:

s					e
2	4	6	7	9	11
0	1	2	3	4	5

- start = 0, end = n-1 (5), target = 7
- on first iteration:

mid = start + (end - start)/2
= 0 + 5/2 → 2

on arr[2] = 6 is 6 < 7 :
start = mid + 1 → start = 3, end = 5

- on Second Iteration :

start = 3 < end = 5

∴ mid = start + (end - start) / 2

= 3 + (5 - 3) / 2

= 3 + 2 / 2 → 1

= 3 + 1 → [4]

∴ mid = 4.

arr[4] = 9 [> 7]

∴ start = 3, end = mid - 1

= 4 - 1

= 3

- on third Iteration :

start = 3 == end = 3

∴ mid = start + (end - start) / 2

= 3 + (3 - 3) / 2

= 3 + 0

∴ mid = 3

arr[3] = 7

& arr[3] == target

∴ We got our target on index '3'.